Finanziato dall'Unione europea
NextGenerationEU

Ministero dell' Istruzione, dell'Università e della Ricerca

EKEEL

EmpoweringKnowledgeExtractiontoEmpowerLearners

# EKEEL Video Augmentation - Technical and User Manual

Developed within the PRIN 2022 PNRR project:
**EKEEL – Empowering Knowledge Extraction to Empower Learners**

# Project Information

This software has been developed within the research project **EKEEL – Empowering Knowledge Extraction to Empower Learners** (PRIN 2022 PNRR).

**Project Title:** Empowering Knowledge Extraction to Empower Learners
**Project Acronym:** EKEEL
**Delivery Date:** May 2025
**Software Name:** EKEEL Video Augmentation

**Developers:** Grzegorz Mikulski, Mario Vallarino

**Scientific Coordinator**: Ilaria Torre

**Disclaimer:** This software has been developed within the PRIN 2022 PNRR project funded by the Italian Ministry of University and Research (MUR) under the European Union – Next Generation EU programme. The views expressed are those of the authors and do not necessarily reflect those of the funding authorities.

**Version History:**

- **v1.0 (Sept 2024):** first release of the software.
- **v2.0 (May 2025):** final release of the software.

# Overview and context

**EKEEL Video Augmentation** is a virtual reality application developed within the EKEEL project. The software contributes to the implementation of the video augmentation service described in Deliverable D5.1 and aligns with the design requirements defined in the project documentation.

This application is designed to support learners by enriching educational videos with services that make conceptual dependencies explicit and highlight prerequisite concepts required to understand new concepts.

The application uses as input a graph representing the concepts explained in a video lesson and their prerequisite relations. The graph is encoded in JSON-LD, which follows semantic standards to represent the concepts, prerequisite links, and timestamped references to video segments associated with each concept (i.e., when a concept is introduced and further explained).

Within the EKEEL project, the graph can be created either manually, with semi-automatic features, using the **EKEEL Annotator**, or using an automated process using the **EKEEL Extractor**.

EKEEL Video Augmentation augments a video lecture by creating an **immersive environment** organized into

three progressive levels:

1. **Augmented first viewing:** The video lesson is viewed in an immersive VR environment, augmented with concept materialization represented as spheres that emerge from the video screen when concepts are introduced, and spatial anchoring to increase salience. A progressively built concept map represents the concepts and their prerequisite relations, establishing the conceptual scaffold for the next levels.

2. **Selective rewatch and exploration:** Learners engage with the educational video through the immersive concept map, which supports navigation of concepts and exploration of their structure and dependencies. Each concept sphere includes an associated panel that allows a finer rewatch of video segments linked to the concept, tracking the multiple occurrences and their progression throughout the video.

3. **Task-oriented embodied activities:** Structured exercises and kinesthetic interactions leverage the graph. Two main activities are included in the released application. In the first activity, the concept map displays only the nodes, and learners must insert the prerequisite relations between concepts themselves. In the second activity, the concept map is complete, but the spheres representing the concepts do not show the concept names. Learners must fill in the missing labels, corresponding to an embodied “fill-the-gap” exercise.

The software is implemented as a **modular VR application**, supporting multiple configurations without requiring separate installations. System behavior is determined by the **selected Operation Mode**, which specifies the active features and runtime parameters for each module (e.g., enabling or disabling interaction features, logging policies, and evaluation routines).

**Modes of Operation (overview)**

**Mode 1**: Interactive concept spheres connected by arrows; the concept in focus is highlighted, and previous concepts change color when a new one comes into focus. Spheres are movable, and interaction allows access to the corresponding video segment.

**Mode 2**: Concepts appear as spheres emerging from the video as they are explained; connections are created progressively. Interaction is the same as Mode 1.

**Mode 3**: Spheres are created while connections are hidden. Learners create the connections themselves and receive feedback on correct or incorrect prerequisite relations.

**Mode 4**: Learners are asked to label the spheres themselves, corresponding to a fill-the-gap activity on concept names.

# Installation manual

This manual shows the main requirements needed to run the unreal engine editor using virtual reality, the setup of the headset itself is omitted.

## Table of contents


1. Requirements
2. Hardware requirements
3. Installation and setup
    i. Installing Meta Quest App
    ii. Installing Unreal Engine
    iii. Launching projects


## 1 Requirements:

- Epic Games account
- Meta account
- Meta Quest App

- Unreal Engine 5.3.2
- Blueprint FileSDK plugin
- VaRest plugin

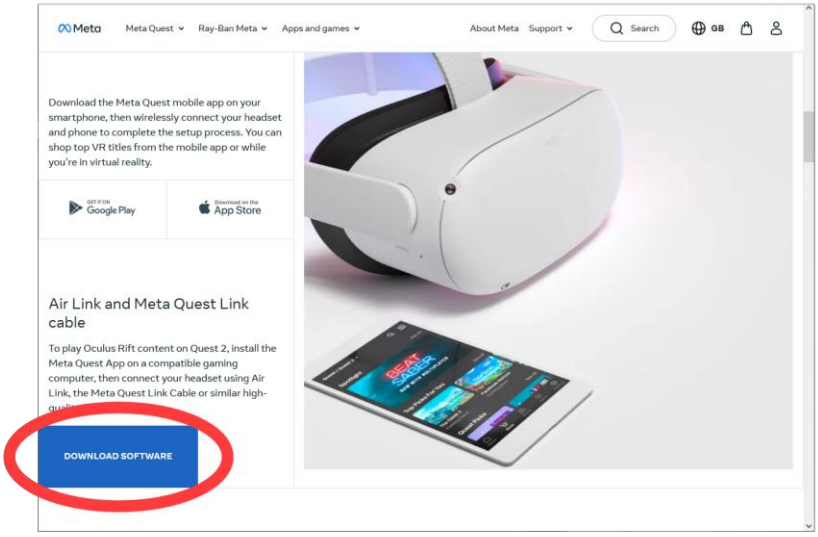
## 2 Hardware requirements:

- Oculus/Meta Quest 2/ Meta Quest 3
- GPU able to handle VR (e.g. GTX 1080)
- CPU able to handle VR (e.g. i7 7700K)

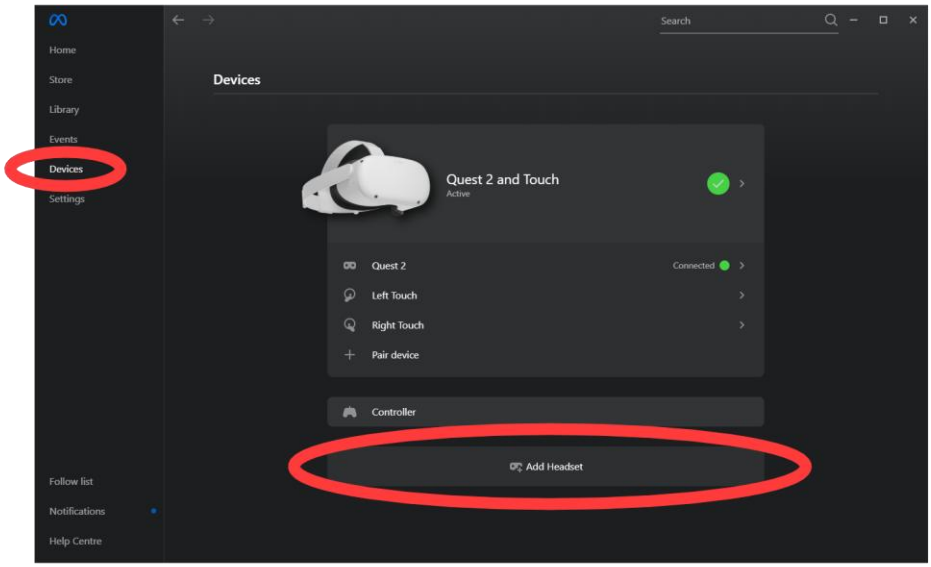
# 3 Installation and setup:

## 3.1 Installing Meta Quest App

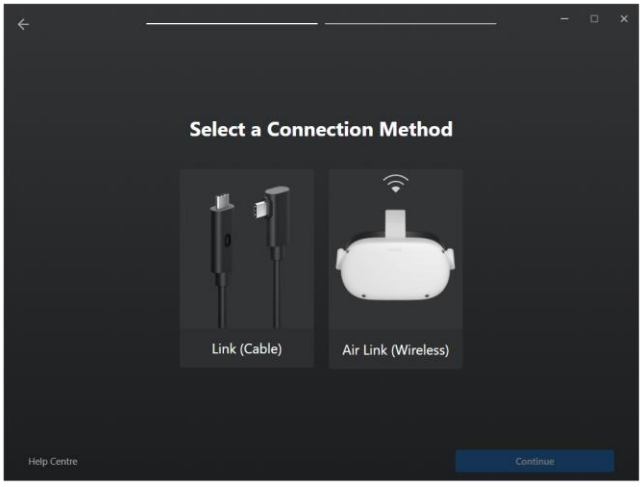
- Download Meta Quest App from [Meta Quest Website]




- Install the application by following the setup wizard.
- Open the Meta Quest App and log in using the Meta account.
- Add the Quest 2 headset to the Meta Quest App by going to the Devices section on the left side bar and pressing "Add Headset".



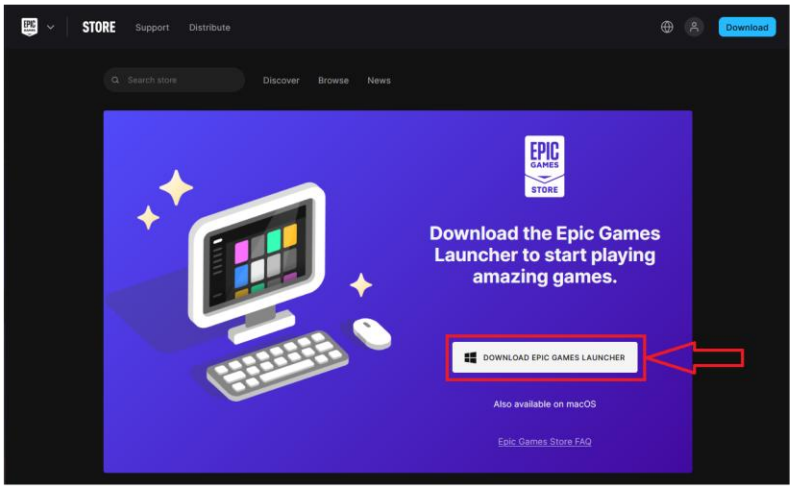

- Connect the Quest headset to the computer via cable (Quest Link) or wifi (Quest Link) by using the Meta Quest App.



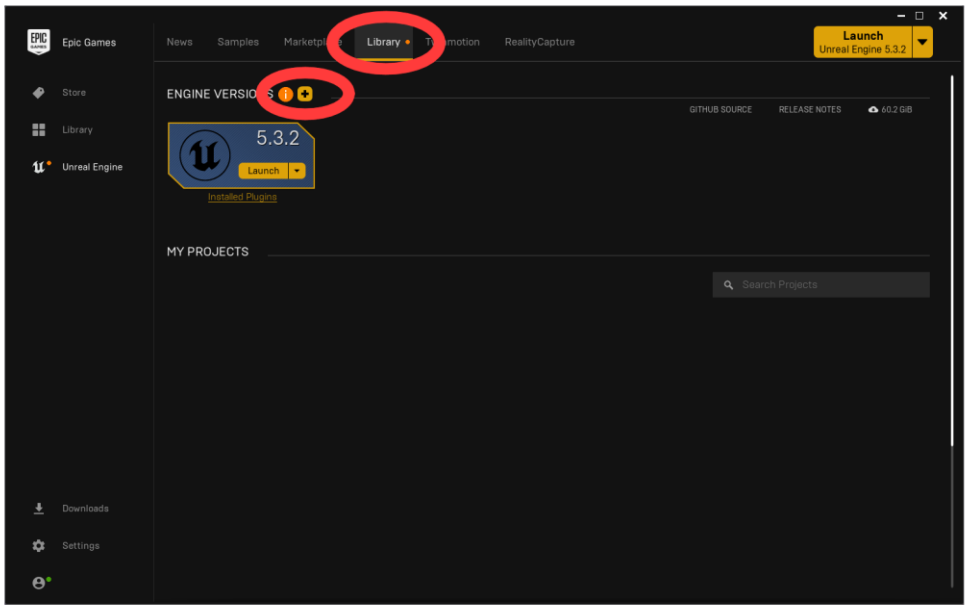
## 3.2 Installing Unreal Engine

- Visit Epic Games and create an account by going into the profile section, when prompted to login press “Create account” and follow the instructions given by the website.
- Download the Epic Games launcher used to access Unreal Engine by visiting the

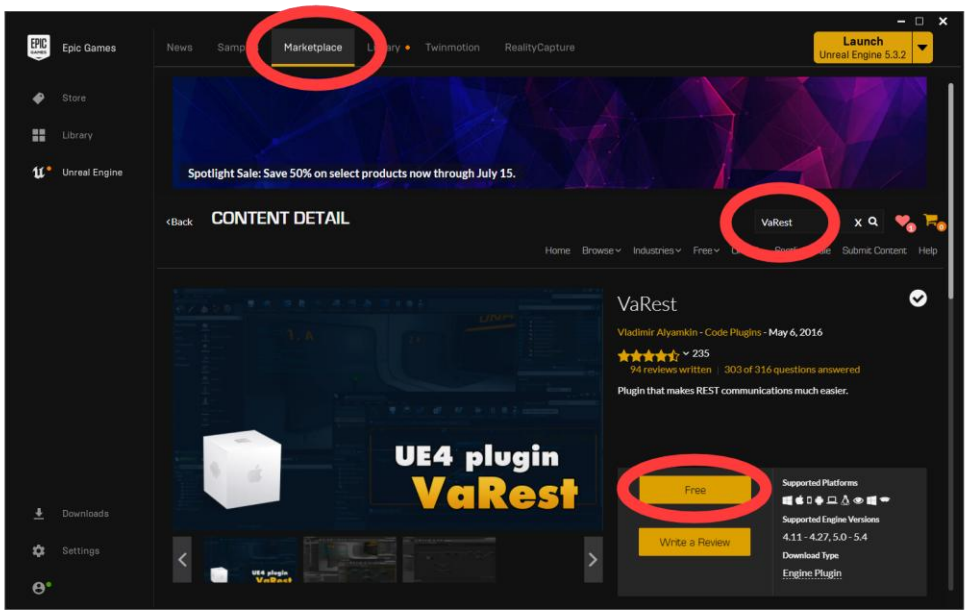
  Epic Games download section




- After download completes install the Epic Games launcher in the desired directory by following the installation wizard.
- Open the Epic Games launcher and log in with the previously created account.
- Go to the Unreal Engine section on the side bar.
- Go to the Library tab on the navigation bar on the upper section.
- Press the "+" icon to add a new engine instance and select version 5.3.2.
- Install Unreal Engine version 5.3.2 in the desired directory.




- Go to the Marketplace tab on the navigation bar on the upper section.
- Browse content for '**Blueprint FileSDK**' and add it to the account.
- Browse content for '**VaRest**' and add it to the account.




- Return to the Library tab to install plugins to instances of engine versions.
- Install '**Blueprint FileSDK**' and '**VaRest**' to the previously installed engine version.

## 3.3 Launching projects

- Open **'EKEEL_Project.uproject'** to open the unreal engine editor with the project.

# Compilation manual

This manual shows how the classes and various assets can be compiled.

## Table of contents

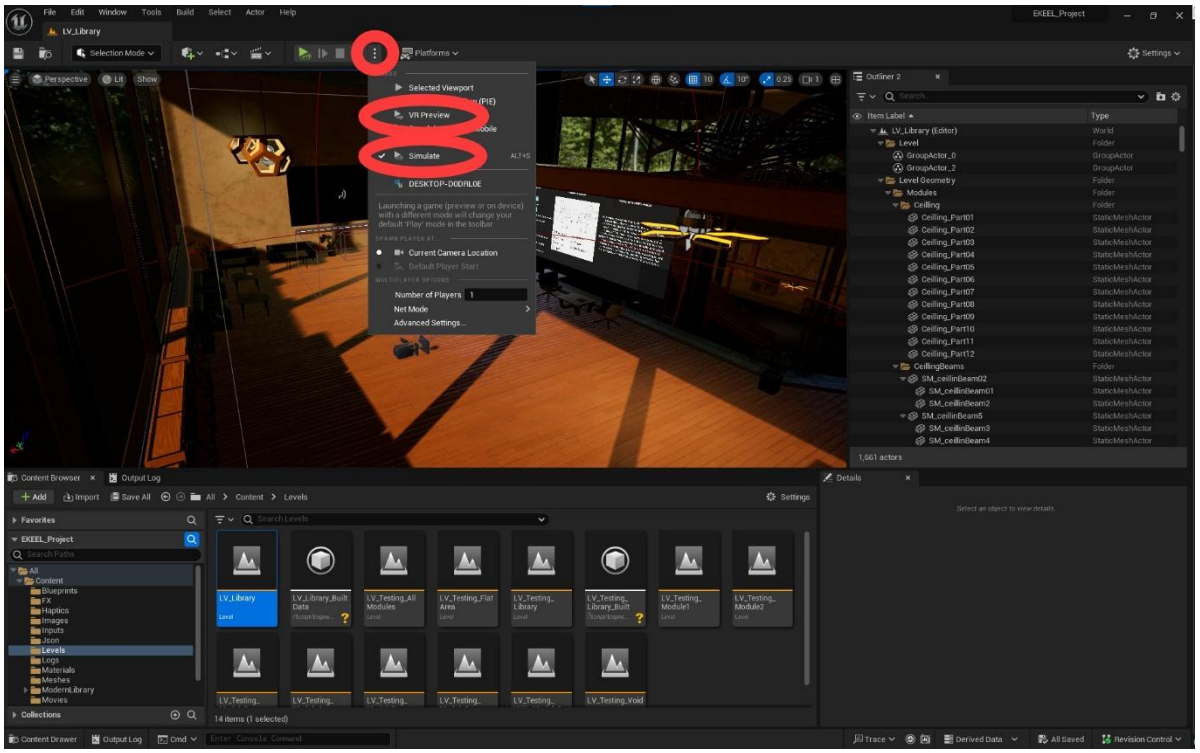
1. Compilation by execution
2. Compiling blueprint classes
3. Compiling into own process
4. Building lighting

## 1 Compilation by execution:

After the project has been opened into Unreal Engine, it is automatically compiled when
running a simulation or VRPreview, the application will run in the Editor
only if no compilation errors occour in the code and/or blueprint
classes, this will run the selected level:
Follow these steps to compile and run the application through VRPreview or Simulation:

- Open EKEEL_Project.uproject

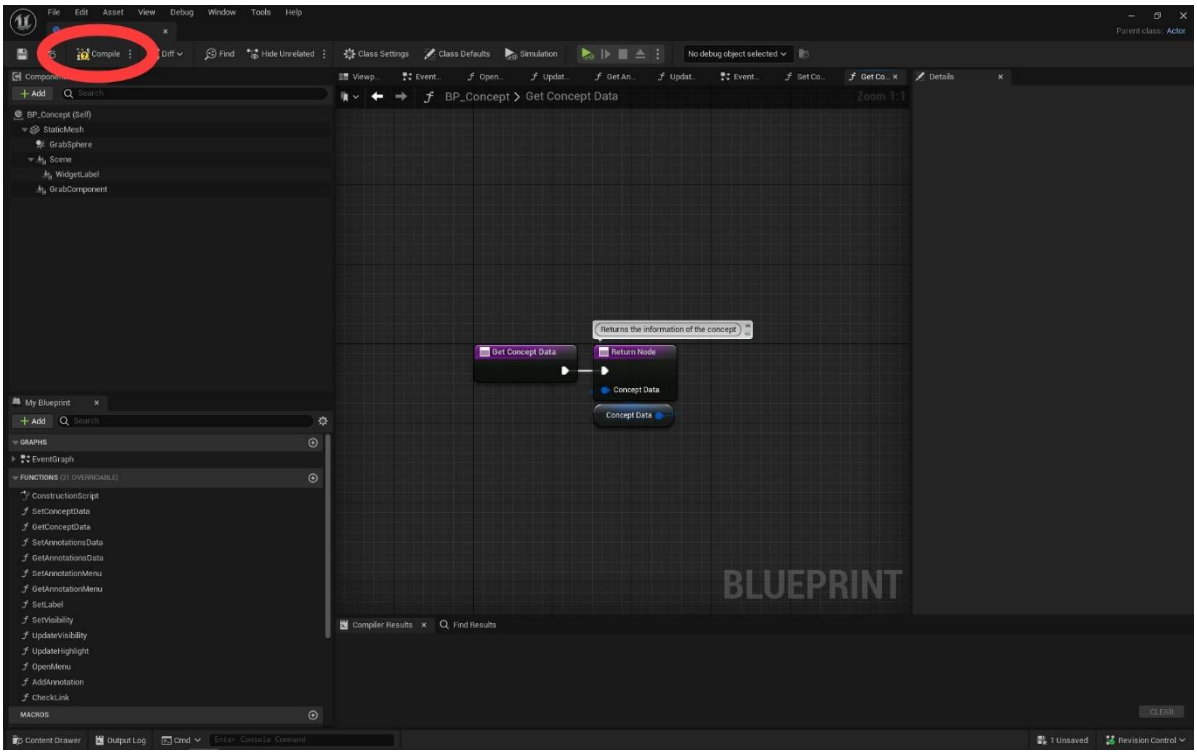
- Open the Levels directory in the content browser
- Open LV_Library (or LV_Testing_AllModules)
- Navigate to the three dots on the upper section of the Editor
- Click and select VRPreview or Simulate




## 2 Compiling blueprint classes:

Blueprint classes can be compiled after some modifications to the code execution have
been made, a class can be compiled by opening the Blueprint editor:

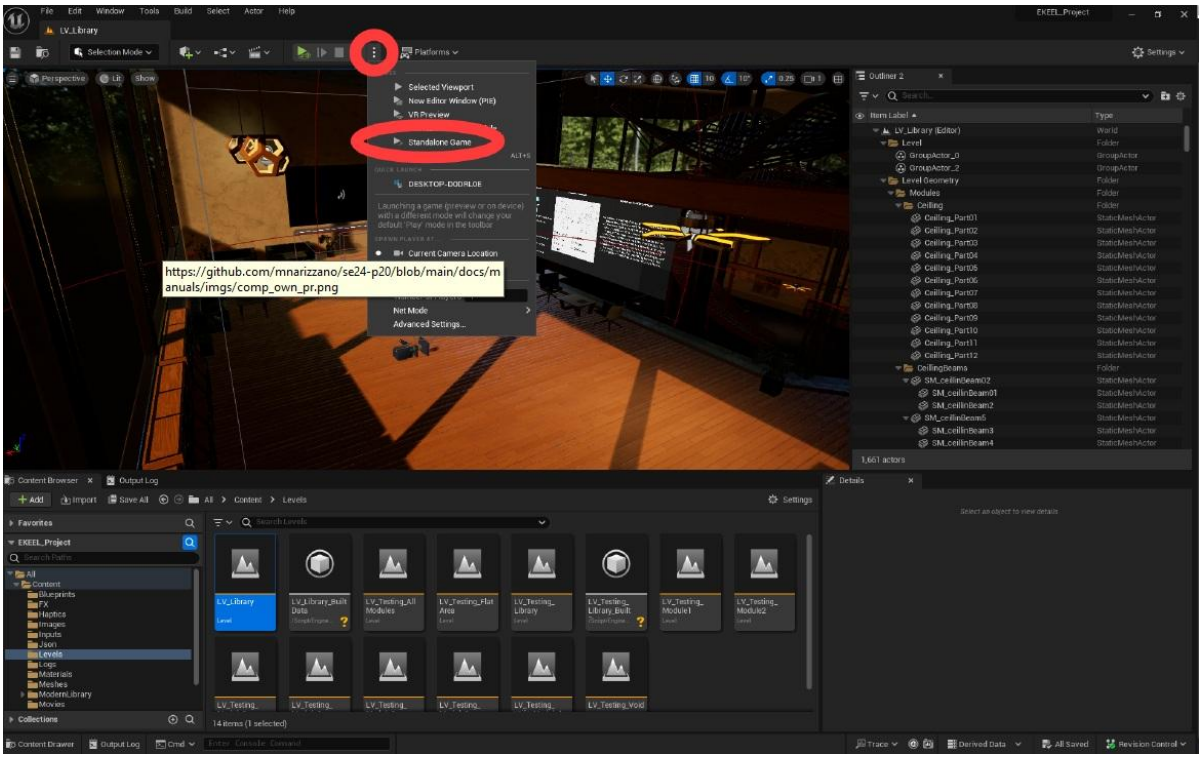
- Open EKEEL_Project.uproject
- Open the Blueprints directory in the content browser
- Double click to open the Blueprint Editor for a class
- On the upper left corner of the Editor press Compile




# 3 Compiling into own process:

The system can be made to be executed in it's own process separate from the Editor, this will run the selected level:

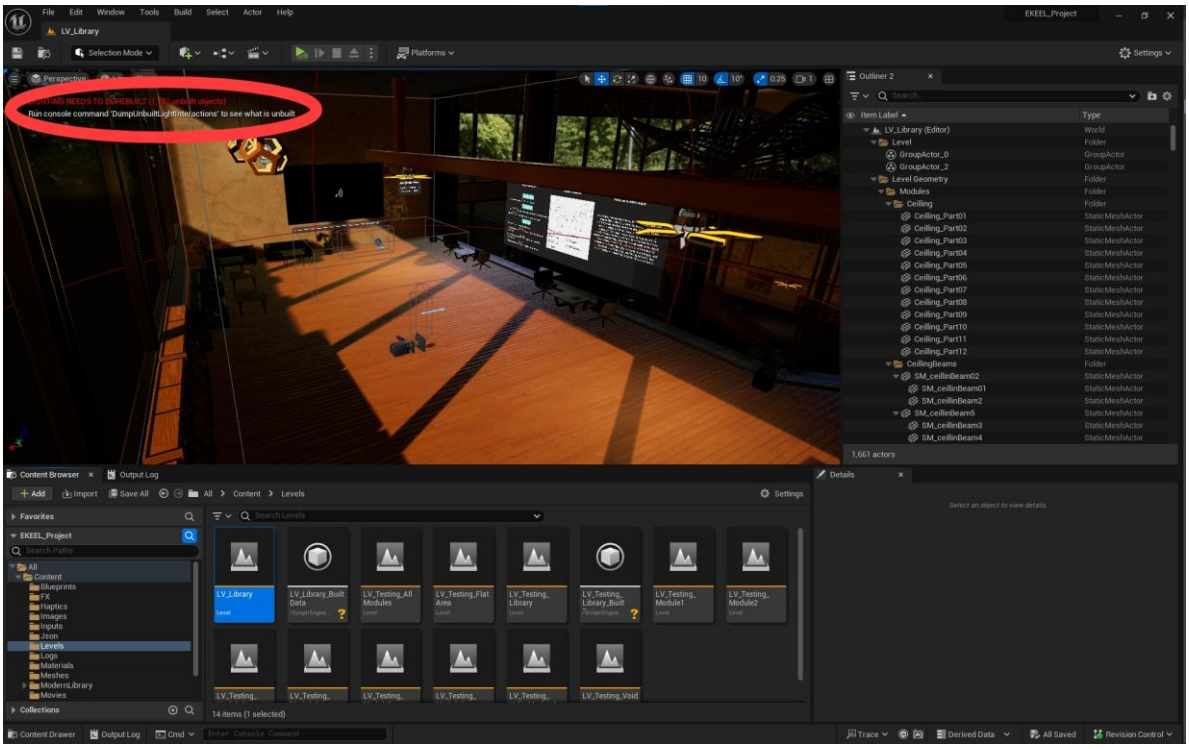
- Open EKEEL_Project.uproject
- Open the Levels directory in the content browser
- Open LV_Library (or LV_Testing_AllModules)
- Navigate to the three dots on the upper section of the Editor
- Click and select StandaloneGame



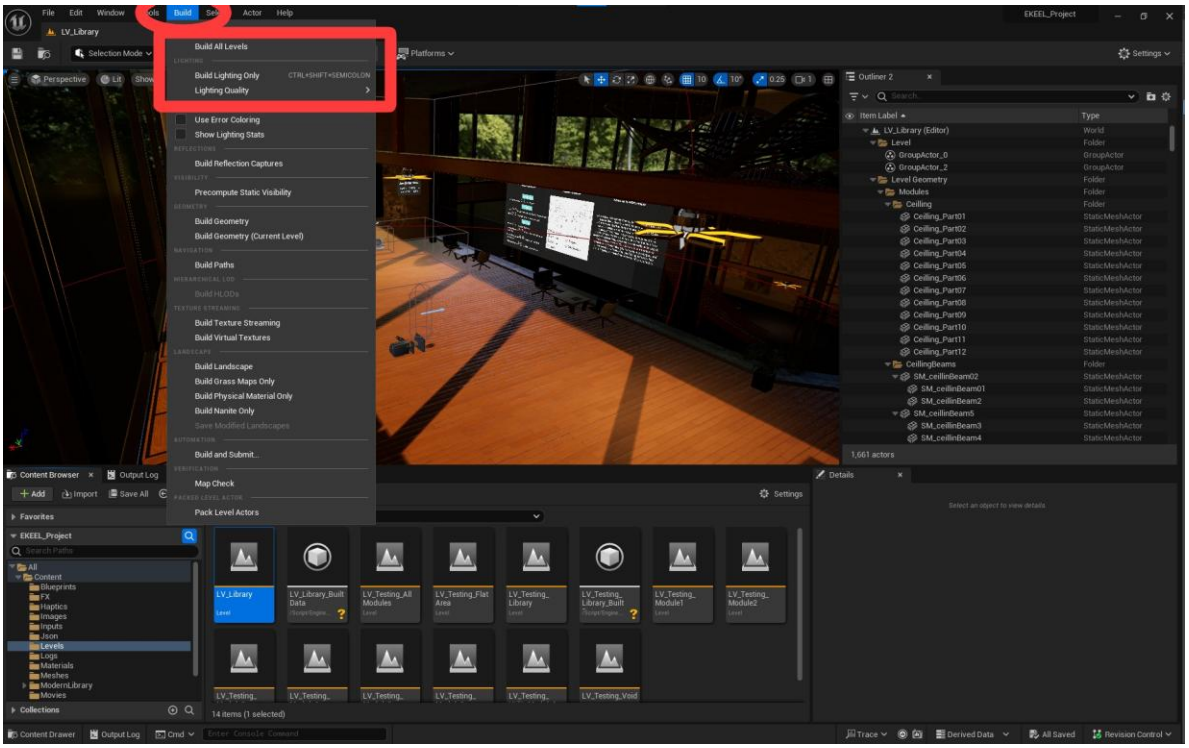

# 4 Building lighting:

If opening levels that have particular lighting assets, such as static lighting by lights onto
the meshes present in the level, a warning message may appear telling
the lighting needs to be built, while the application will run regardless of
this, to resolve the warning follow these steps:

- Open EKEEL_Project.uproject
- Open the Levels directory in the content browser
- Open LV_Library
- Verify the presence of the Lighting Requires Building warning




- On the navigation bar of Unreal Engine select Build
- Select Lighting Quality to specify the quality of the lighting (the execution time depends on the performance of the processor)
- Select Build Lighting Only or Build All Levels


File
Edit
Window
Build
Actor
Help
LV_Library
EKEEL_Project
Build All Levels
Build Lighting Only
CTRL+SHIFT+SEMICOLON
Lighting Quality
Use Error Coloring
Show Lighting Stats
Build Reflection Captures
Precompute Static Visibility
Build Geometry
Build Geometry (Current Level)
Build Paths
Build HLODs
Build Texture Streaming
Build Virtual Textures
Build Landscape
Build Grass Maps Only
Build Physical Material Only
Build Nanite Only
Save Modified Landscapes
Build and Submit...
Map Check
Pack Level Actors
Platforms
Settings
Outliner 2
Item Label
Type
LV_Library (Editor)
Level
Level Geometry
Modules
Ceiling
1,661 actors
Content Browser
Output Log
Add
Import
Save All
Favorites
EKEEL_Project
Content
Blueprints
FX
Haptics
Images
Inputs
Levels
Logs
Materials
Meshes
ModernLibrary
Movies
Collections
LV_Library
LV_Library_Built Data
LV_Testing_All Modules
LV_Testing_Flat Area
LV_Testing_Library
LV_Testing_Library_Built
LV_Testing_Module1
LV_Testing_Module2
LV_Testing_Void
14 items (1 selected)
Details
Content Drawer
Output Log
Cmd
Enter Console Command
Trace
Derived Data
All Saved
Revision Control

# Run manual

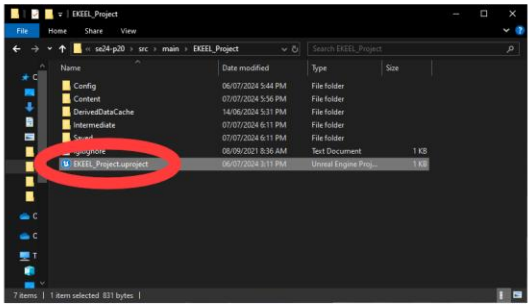
## Table of contents


1. Opening the project
2. Running application in VR mode
3. Running application in Simulation mode
4. Setting Mode of Operation
5. Adding custom concept map files
    i. Adding json files
    ii. Adding video files
6. Setting which files to be used
    i. Setting path of json data files
    ii. Setting path of video data files
    iii. Setting path of log data files
7. Toggling Link name visibility

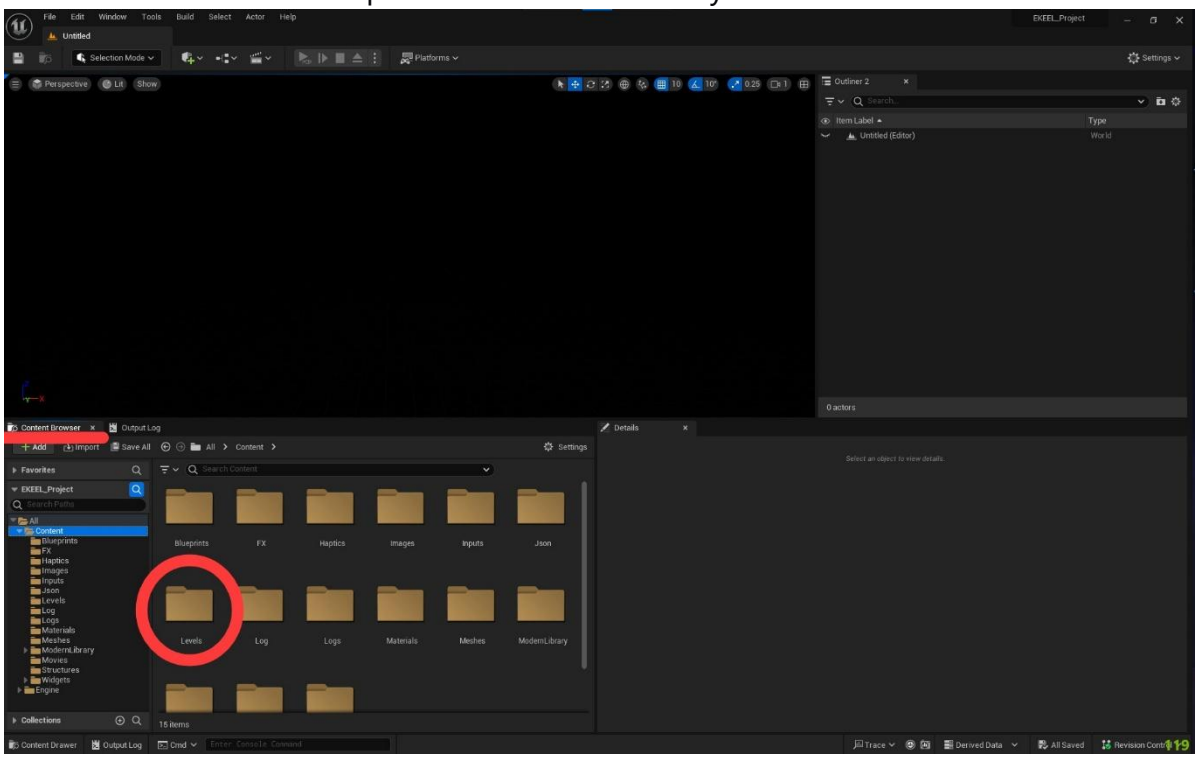

## 1 Opening the project:

Follow these steps to open the application with the provided environment using the default settings.

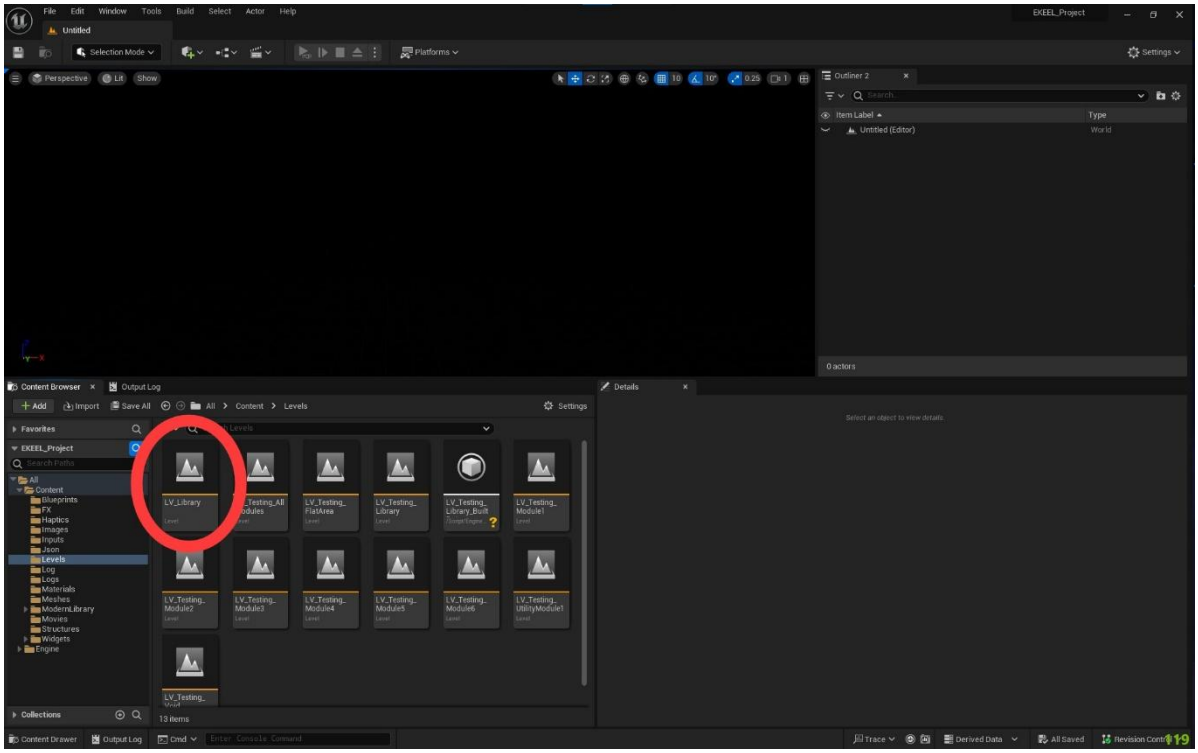
- Open the project by running " `EKEEL_Project.uproject` ".




- In the `Content Browser` open the `Levels` directory.



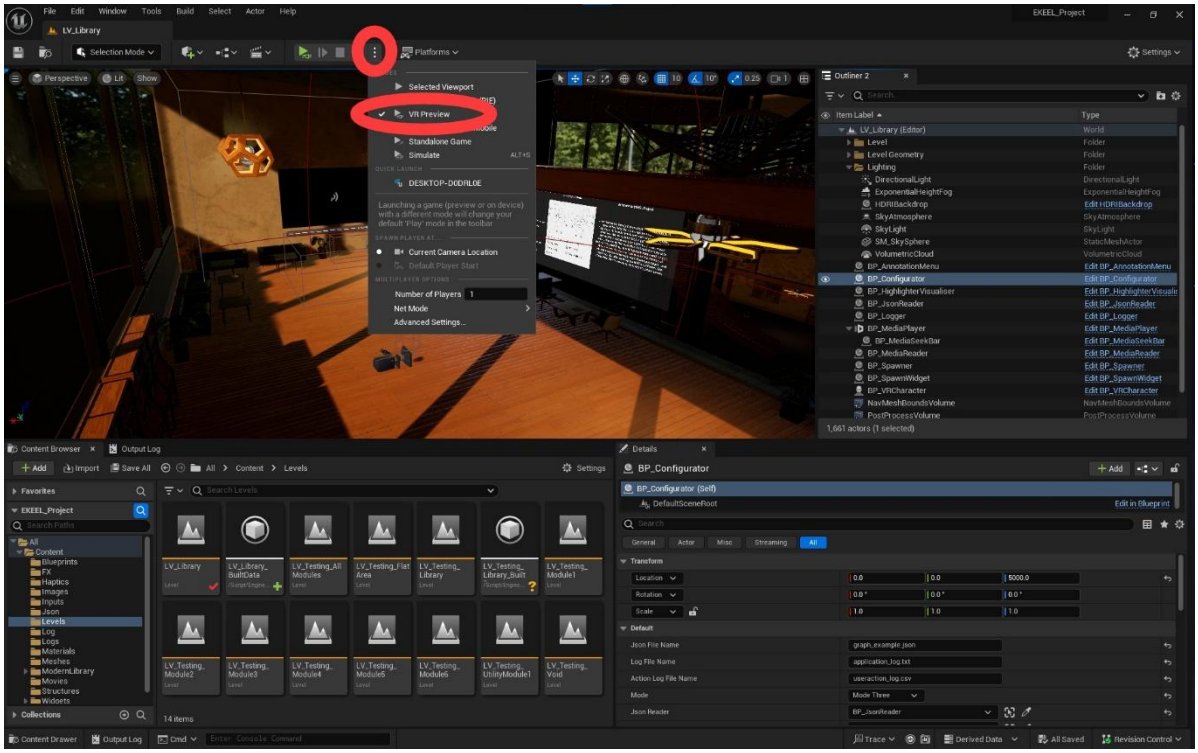
- Open the `LV_Library` level to use the application in the preconfigured Library environment.




## 2 Running application in VR mode:

This will run the application using the virtual reality mode on the currently open level, this requires the Quest 2 headset (preferred) to be connected to the Meta Quest App, both the Meta Quest App and Unreal Engine must be running.

- Open the run options by pressing the three dots on the upper menu in unreal engine.
- If the headset is connected the " `VR Preview` " option should be selectable.

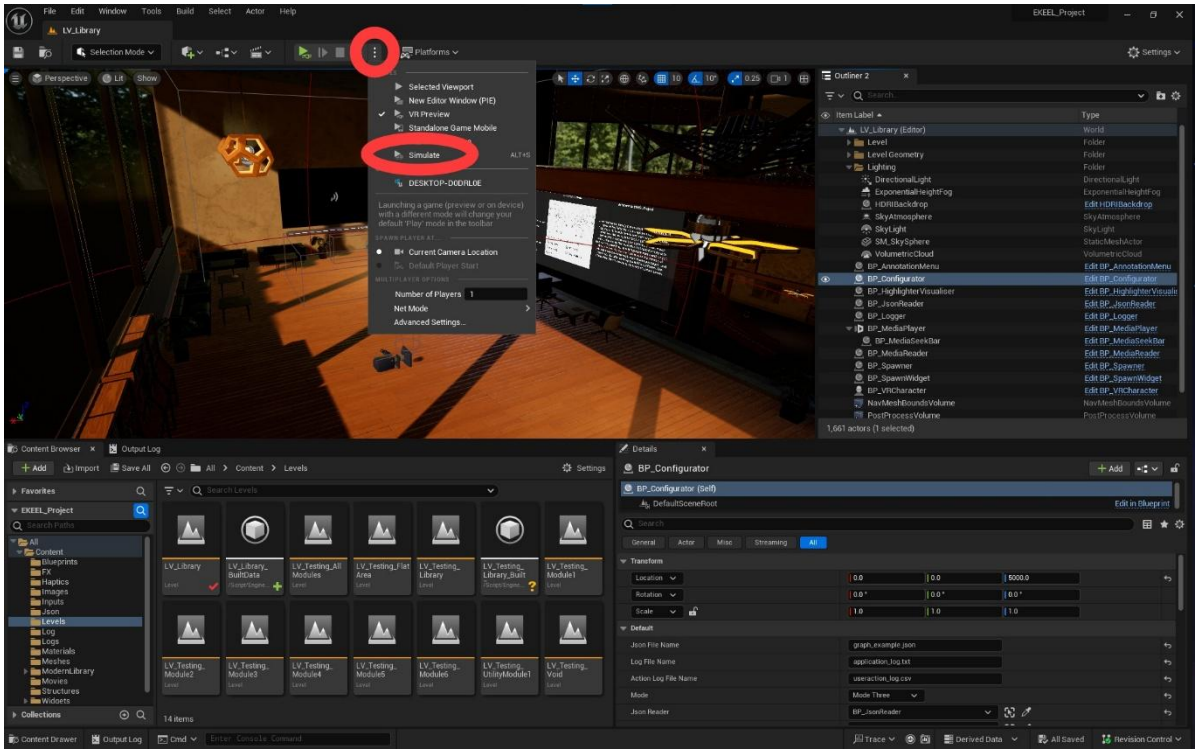



- Put on the headset and use the application.

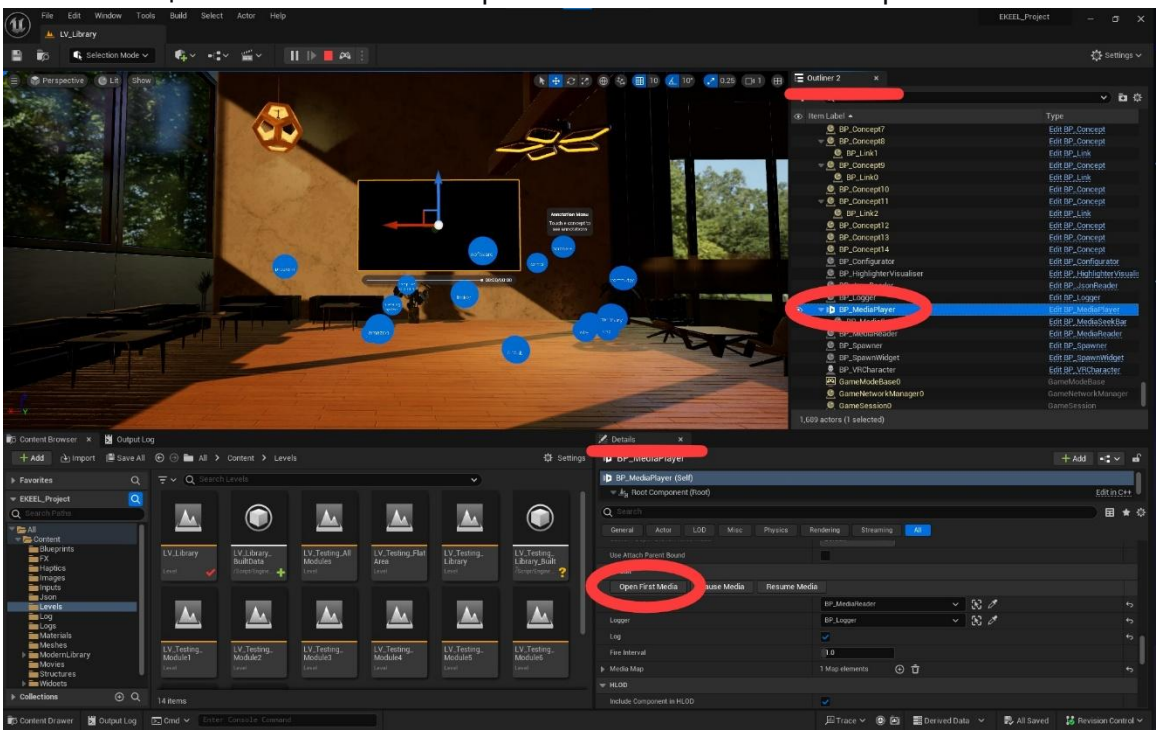
# 3 Running application in Simulation mode:

This will run the application using the simulation mode on the currently open level, note that this mode is mainly used for debugging since the interactivity is severely limited to the callable functions of the individual modules, the concepts and links will be generated based on the mode of operation.

- Open the run options by pressing the three dots on the upper menu in unreal engine.
- Select " `Simulate` "

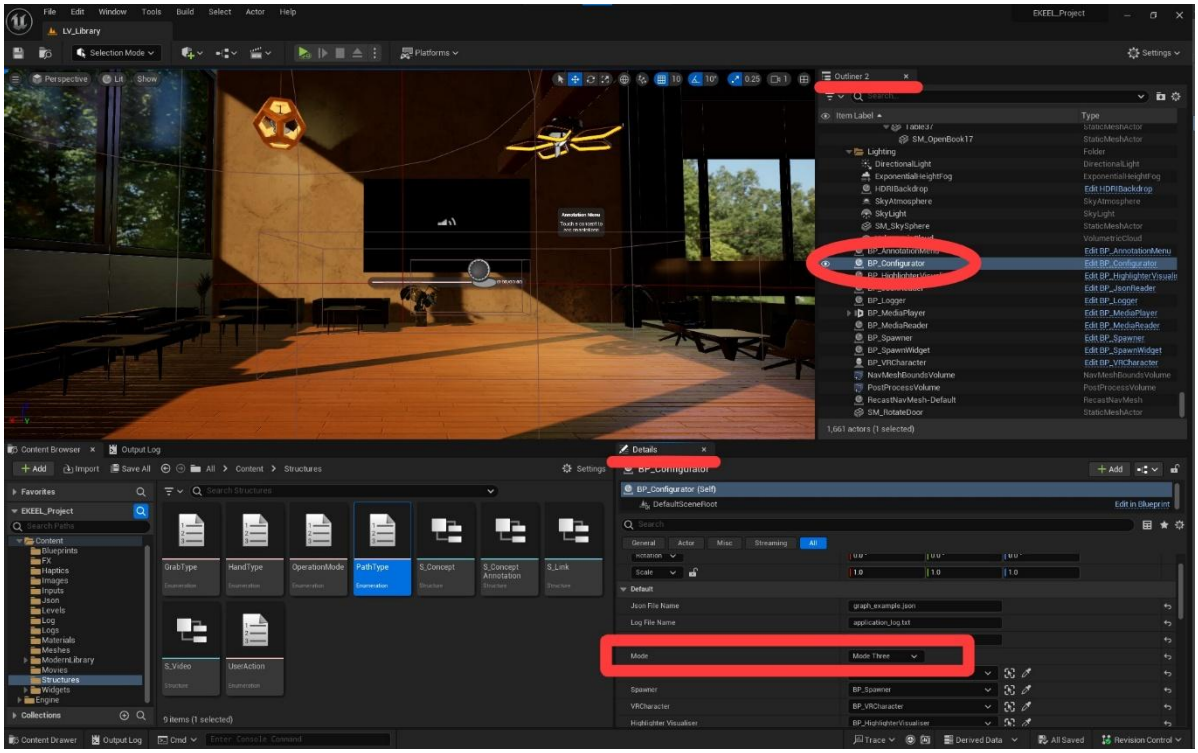



- Move around the virtual area by holding the right mouse button and using WASD keys to move.
- Click the `BP_MediaPlayer` object in the root of the level Outliner.
- Scroll down to the " `Default` " section in the " `Details` " panel.
- Press on " `Open First Media` " to open the video from the start point.



# 4 Setting Mode of Operation:

- Click the `BP_Configurator` in the root of the level Outliner.
- Go to the " `Default` " category in the " `Details` " panel.
- Set the " `Mode` " parameter to one of the available modes.




# 5 Adding custom concept map files:

This section provides a guide to add new concept map data to be read by the application.

## 5.1 Adding json files:

- Json files must have a valid json syntax and follow the proper data model in order for the files to be read correctly. The data model is mainly based on the Web Annotation Ontology, extended with some classes and properties belonging to the Edurell Vocabulary, and others specifically defined for the Ekeel data model.

- Place the .json file in the `EKEEL_Project/Content/Json/` directory.

NOTE: Json files in other directories are still usable but the modules have to be set to read accordingly, refer to section 6.1

## 5.2 Adding video files:

- Video files must be named according to the .json file annotation references (e.g. the video name must match the annotation source).
- Place the video file (preferred .mp4 format) in the `EKEEL_Project/Content/Movies/` directory.

NOTE: Video files in other directories are still usable but the modules have to be set to read accordingly, refer to section 6.2

# 6 Setting which files to be used:

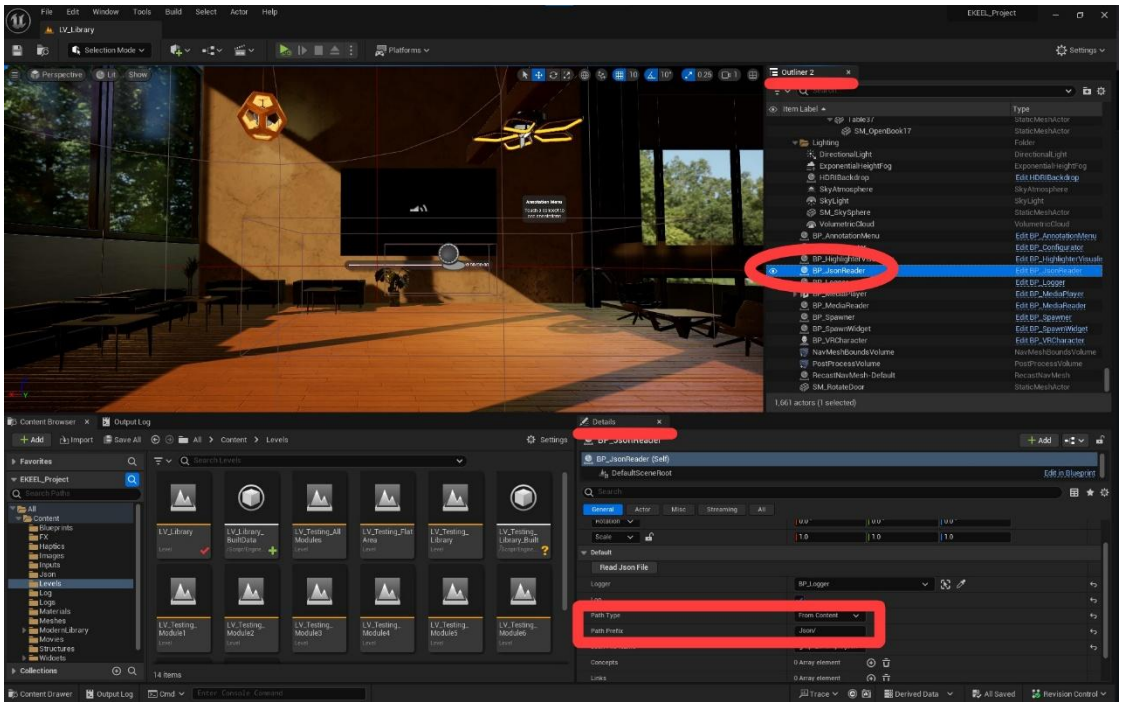
This section provides a guide to set the modules to read files in different directories.

## 6.1 Setting path of json data files:

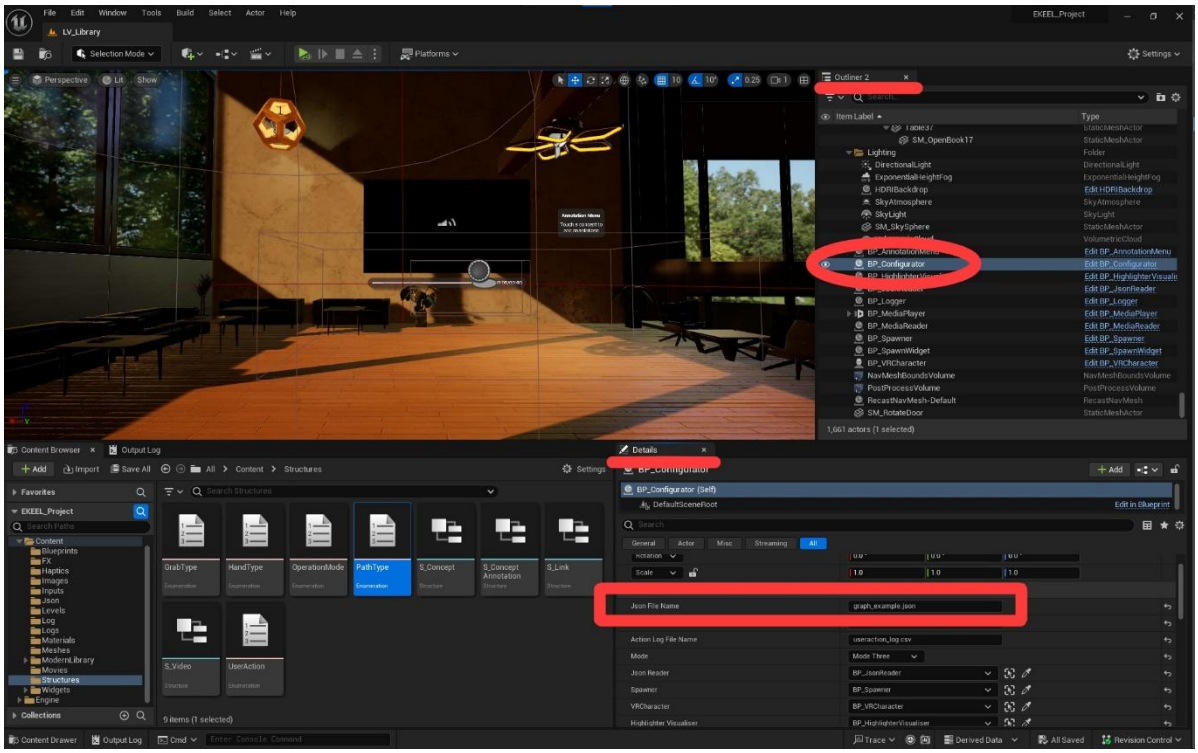
- Click the `BP_JsonReader` in the root of the level Outliner.
- Go to the " `Default` " category in the " `Details` " panel.
- Set the `PathType` and `PathPrefix` values:
  - " `From Content` " will automatically load the path to the Content folder, if needed add a prefix to further specify the file location folder (i.e. `From Content + "Json/"` )
  - " `From Directory` " will automatically load the path to the Unreal Engine project directory ( `EKEEL_Project/` ) if needed add a prefix to further

specify the file location folder (i.e. `From Directory + "Content/Json/"` )

- " `Full Path` " requires to set the complete path to the directory the file is contained in as the PathPrefix field (i.e. `"C:/.../EKEEL_Project/.../" + "File.json"` )



- Click the `BP_Configurator` in the root of the level Outliner.
- Go to the " `Default` " category in the " `Details` " panel.
- Set the " `JsonFileName` " file name field to be read including the extension.

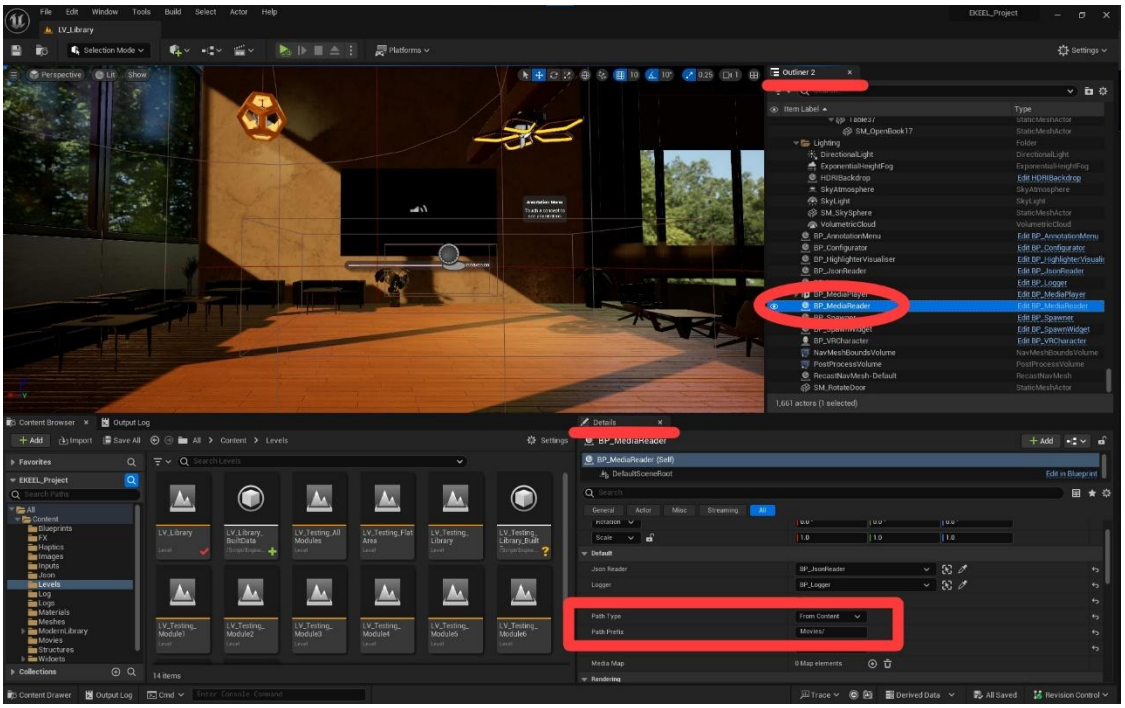


NOTE: The name of the file is not needed to be set on the BP_JsonReader module if it's linked to the BP_Configurator

## 6.2 Setting path of video data files:

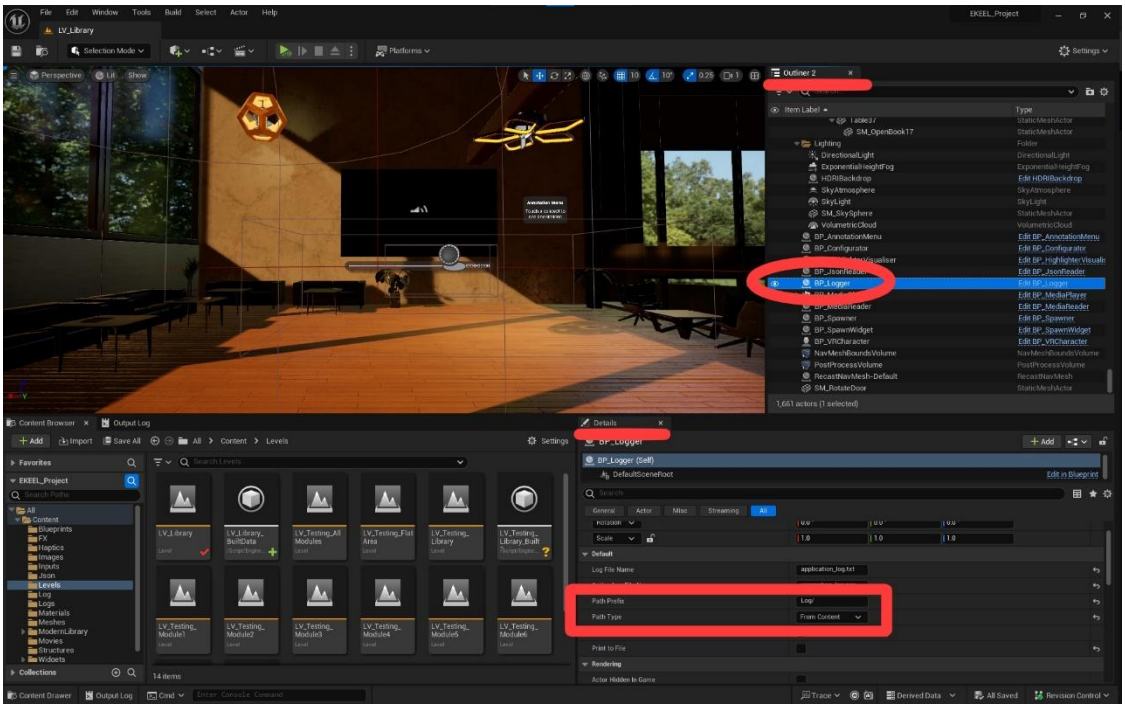
- Click the `BP_MediaReader` in the root of the level Outliner.
- Go to the " `Default` " category in the " `Details` " panel.
- Set the `PathType` and `PathPrefix` values:
  - " `From Content` " will automatically load the path to the Content folder, if needed add a prefix to further specify the file location folder (i.e. `From Content + "Movies/"` )
  - " `From Directory` " will automatically load the path to the Unreal Engine project directory ( `EKEEL_Project/` ) if needed add a prefix to further specify the file location folder (i.e. `From Directory + "Content/Movies/"` )

- " `Full Path` " requires to set the complete path to the directory the file is contained in as the PathPrefix field (i.e. `"C:/.../EKEEL_Project/.../" + "File.mp4"` )



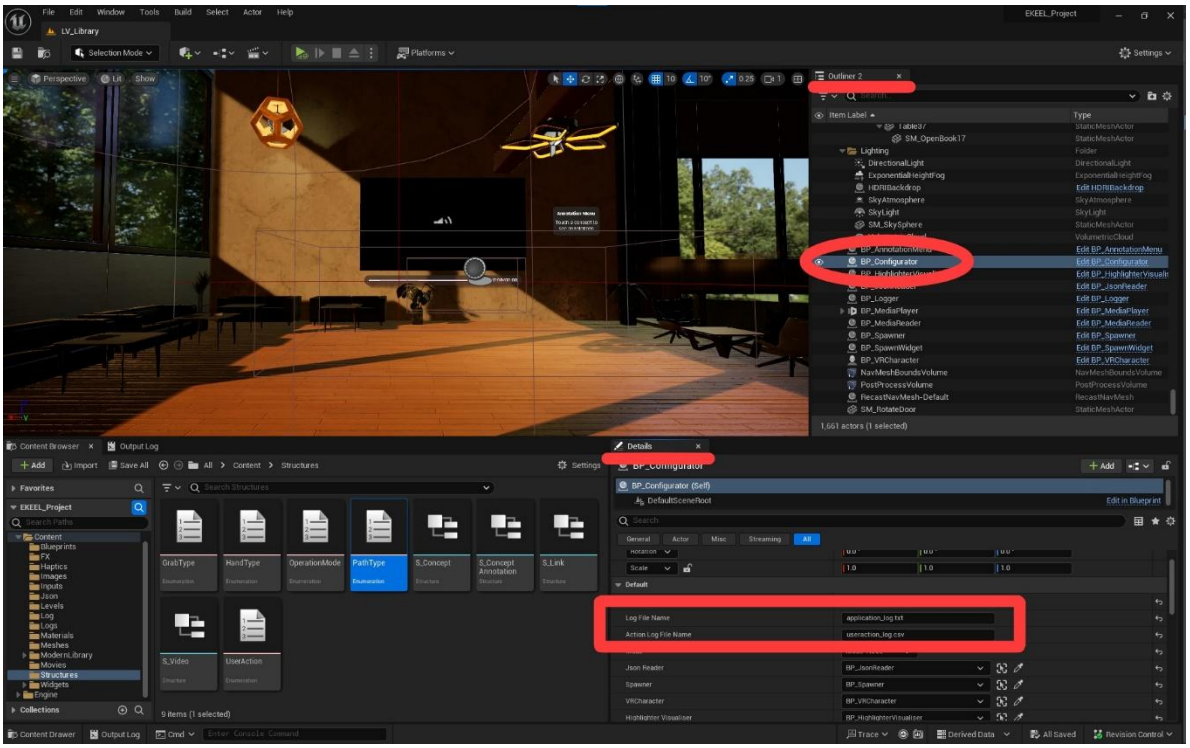
## 6.3 Setting path of log data files:

- Click the `BP_Logger` in the root of the level Outliner.
- Go to the " `Default` " category in the " `Details` " panel.
- Set the PathType and PathPrefix values:
  - " `From Content` " will automatically load the path to the Content folder, if needed add a prefix to further specify the file location folder (i.e. `From Content + "Logs/ "`)
  - " `From Directory` " will automatically load the path to the Unreal Engine project directory ( `EKEEL_Project/` ) if needed add a prefix to further specify the file location folder (i.e. `From Directory + "Content/Logs/"` )
  - " `Full Path` " requires to set the complete path to the directory the file is contained in as the PathPrefix field (i.e. " `C:/.../EKEEL_Project/.../" + "File.txt` ")



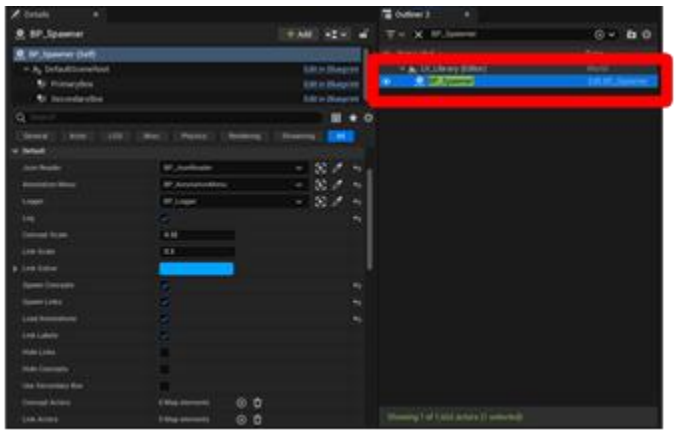
- Click the `BP_Configurator` in the root of the level Outliner.
- Go to the " `Default` " category in the " `Details` " panel.
- Set the " `LogFileName` " field to be written to including the extension for the application module messages (i.e. " `app_log.txt` ").
- Set the " `ActionLogFileName` " field to be written to including the extension for the user actions (i.e. " `action_log.csv` ").

NOTE: The name of the files is not needed to be set on the BP_Logger module if it's linked to the BP_Configurator

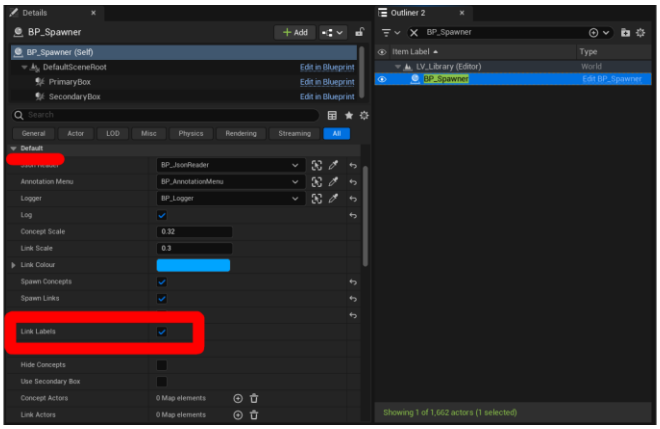


# 7 Toggling Link name visibility:

- Make sure the Simulation or VRPreview is not running.
- Click the `BP_Spawner` in the root of the level Outliner.



- Go to the " `Default` " category in the " `Details` " panel.
- Enable or Disable the 'Link Labels' option to make labels visible or not.




- Run the application.